

Challenges

Challenges

Four incidents from this build that showed real engineering judgment.

Loki crash-looped: disabling persistence doesn't give it an emptyDir

I added Loki following the same pattern as Prometheus: disable persistence, assume the chart falls back to an emptyDir. It failed with “context deadline exceeded” after 10 minutes, and `atomic = true` auto-uninstalled it before I could inspect anything.

I reproduced it manually with `--atomic/--wait off` so a failing pod would stay up. It crash-looped with `mkdir /var/loki: read-only file system, error initialising module: ruler-storage`. The cause: the container runs `readOnlyRootFilesystem: true`, and `persistence.enabled: false` just omits the `/var/loki` volume entirely rather than substituting an emptyDir. The ruler tries to `mkdir` its WAL directory there unconditionally on startup.

I confirmed the fix's shape with `helm template ... | grep -A5 /var/loki` before applying anything: an explicit `extraVolumes/extraVolumeMounts` pair produces a real emptyDir mount, not a PVC. Re-installed manually the same way; the pod reached 2/2 Running in 47 seconds with zero restarts.

atomic = true on a first-time install destroys the evidence it needs

The Loki incident above cost its full 10-minute timeout for nothing, because `atomic` rolled back and deleted the failed release before I could look at it.

`atomic` is right for the app's CI deploy: a release that has previously succeeded should roll back cleanly on a bad deploy rather than leave a half-updated Deployment serving traffic. It's wrong for an addon install that has never once succeeded, since there's no working prior state to protect and the only thing it accomplishes on a first failure is deleting the one piece of evidence a diagnosis needs.

I set `atomic = false` on that one release, with a comment explaining the distinction so it isn't corrected back later without the context.

Copy-paste bugs in envs/production, caught before any apply

envs/production was scaffolded from envs/staging and inherited two live bugs, found while wiring the production CI/CD job, before the environment was ever applied.

main.tf's access entry referenced staging's CI role and namespace instead of production's own ci_production role and namespace. module.rds still carried staging's nightly-teardown overrides, no backup retention and no deletion protection, on an environment that isn't torn down nightly.

I fixed the role/namespace reference and set deletion_protection/skip_final_snapshot/backup_retention_period to true/false/7. terraform plan confirmed both fixes with no other drift: 78 to add, 0 to change, 0 to destroy.

No ALB target-health metric without more infrastructure

The platform dashboard has no ALB target-health panel, because that metric doesn't exist here. Getting one would mean running a CloudWatch exporter against AWS/ApplicationELB and AWS/TargetGroup, for a platform that deliberately doesn't run that exporter.

I used sum(up{job="app"}) instead, Prometheus's own scrape-health signal for the app's targets, and kube_deployment_status_replicas_available from kube-state-metrics. Neither tells me whether the ALB itself considers a target healthy, only whether the pods behind it are up.